

cs2521

Week 2 | Recursion & Algo Analysis

Ronnie Low

Table of Contents

Table of Contents	1
Agenda	3
Week 2 Agenda	4
Algorithm Analysis	5
How do we analyse algorithms?	6
Input size relation	7
Finding an item in an array	8
Recursion	9
What is recursion	10
Subproblem example	11
Tutorial demo!	12

Agenda

- Algorithm Analysis
 - Time complexity rundown
 - Examples
- Recursion
 - What is recursion
 - Examples

Algorithm Analysis

How do we analyse algorithms?

- How much time an algorithm takes to complete
- How much memory (space) an algorithm needs to complete

- Usually, this is analysed in terms of worst cases!
- We are usually concerned about how the algorithm will perform in its **worst** case
- We usually analyse time and space complexity in regards to the input size

Input size relation

- Our inputs can take on any size
 - If our algorithm works on arrays but arrays can be any size, analysing complexity can be difficult
 - We want to generalise this:
 - We can say that arrays have a size n . Thus, we analyse the complexity in relation to n !
- Lets look at some examples!

- We are tasked with finding an item in an array left to right:

[1, 2, 3, 4, 5, 6, 7]

- What are the best case time complexities and worst case time complexities

- We are tasked with finding an item in an array left to right:

[1, 2, 3, 4, 5, 6, 7]

- What are the best case time complexities and worst case time complexities
 - Best case is when we are searching for 1, since we find it immediately

- We are tasked with finding an item in an array left to right:

[1, 2, 3, 4, 5, 6, 7]

- What are the best case time complexities and worst case time complexities
 - Best case is when we are searching for 1, since we find it immediately
 - worst case is when we are searching for 7, since its at the end.

Finding an item in an array

- We are tasked with finding an item in an array left to right:

[1, 2, 3, 4, 5, 6, 7]

- What are the best case time complexities and worst case time complexities
 - Best case is when we are searching for 1, since we find it immediately
 - worst case is when we are searching for 7, since its at the end.
 - That means that for an array of size n , our time complexity is $O(n)$

Recursion

What is recursion

- A problem solving technique where you solve subproblems
 - a subproblem is when you break a problem down into smaller, easier to solve problems
- How do we break a problem down into a subproblem?

Subproblem example

We are tasked with finding the sum of a linked list. Lets do this on the example linked list below:

$[1] \rightarrow [2] \rightarrow [3] \rightarrow [4] \rightarrow [5]$

- How do we break this down into a smaller problem?

Subproblem example

We are tasked with finding the sum of a linked list. Lets do this on the example linked list below:

$[1] \rightarrow [2] \rightarrow [3] \rightarrow [4] \rightarrow [5]$

- How do we break this down into a smaller problem?
 - The first item + rest of the list: $1 + (\text{rest of the list } 2, 3, 4, 5)$

We are tasked with finding the sum of a linked list. Lets do this on the example linked list below:

$[1] \rightarrow [2] \rightarrow [3] \rightarrow [4] \rightarrow [5]$

- How do we break this down into a smaller problem?
 - The first item + rest of the list: $1 + (\text{rest of the list } 2, 3, 4, 5)$
 - We can then repeatedly solve this for the rest of the list!
 - $2 + (\text{rest of the list } 3, 4, 5)$

```
int sum(struct node *head) {  
    if (head == NULL) return 0;  
    return head->value + sum(head->next);  
}
```

Tutorial demo!
